Linear regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 5, 4, 5])
model = LinearRegression()
model.fit(X, y)
y_pred = model.predict(X)
# Plot
plt.scatter(X, y)
plt.plot(X, y_pred)
plt.title("Linear Regression")
plt.show()
```

Logistic regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
X = np.array([1,2,3,4,5,6,7,8]).reshape(-1,1)
y = np.array([0,0,0,0,1,1,1,1])
model = LogisticRegression()
model.fit(X, y)
X_test = np.linspace(1, 8, 100).reshape(-1,1)
y_prob = model.predict_proba(X_test)[:,1]
plt.scatter(X, y)
```

```
plt.plot(X_test, y_prob)
plt.title("Logistic Regression")
plt.show()
```

Decision Tree

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
data = load_iris()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = DecisionTreeClassifier()
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
plt.figure(figsize=(8,6))
plot_tree(model, filled=True)
plt.show()
```

Naive Bayes (Gaussian)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
data = load_iris()
X = data.data
```

```
y = data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = GaussianNB()
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

PCA (reduce to 2D + plot)

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
data = load_iris()
X = data.data
y = data.target
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.title("PCA - 2D")
plt.show()
```

Random Forest (accuracy)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
data = load_iris()
X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = RandomForestClassifier()
model.fit(X_train, y_train)
print("Accuracy:", model.score(X_test, y_test))
```

Experiment 1 – Linear Regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
x = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)
y = np.array([2, 4, 5, 4, 6])
lr = LinearRegression().fit(x, y)
plt.figure()
plt.scatter(x, y)
plt.plot(x, lr.predict(x), color="red")
plt.title("Linear Regression")
plt.show(block=True)
```

Experiment 2 – Logistic Regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LogisticRegression
x = np.array([[1], [2], [3], [4], [5]])
y = np.array([0, 0, 0, 1, 1])
model = LogisticRegression().fit(x, y)
plt.scatter(x, y, color="blue")
plt.plot(x, model.predict(x), color="red", linestyle="--")
plt.title("Logistic Regression")
plt.show(block=True)
```

Experiment 3 – Decision Tree

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
x = [[0], [1], [2], [3], [4]]
y = [0, 0, 0, 1, 1]
model = DecisionTreeClassifier().fit(x, y)
plt.figure(figsize=(4, 3))
plot_tree(model, filled=True)
plt.show(block=True)
```

Experiment 4 – Naive Bayes Classifier

```
from sklearn.naive_bayes import GaussianNB
import numpy as np
x = np.array([[1], [2], [3], [4], [5]])
y = np.array([0, 0, 1, 1, 1])
```

```
model = GaussianNB()
model.fit(x, y)
print("Predictions:", model.predict([[1], [2], [3], [4], [5]]))
```

Experiment 5 – Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
import numpy as np
x = np.array([[1], [2], [3], [4], [5]])
y = np.array([0, 0, 1, 1, 1])
model = RandomForestClassifier(n_estimators=5)
model.fit(x, y)
print("Predictions:", model.predict(x))
```

Experiment 8 – Hierarchical Clustering

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
x = np.array([[1, 2], [1, 5], [1, 0], [4, 3], [4, 5], [4, 0]])
z = linkage(x, 'ward')
dendrogram(z)
plt.title("Hierarchical Clustering Dendrogram")
plt.show(block=True)
```

Experiment 9 – DBSCAN Clustering

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
x = np.array([[1, 2], [1, 5], [1, 0], [4, 2], [4, 5], [4, 0]])
db = DBSCAN(eps=2, min_samples=2).fit(x)
labels = db.labels_
plt.scatter(x[:, 0], x[:, 1], c=labels)
plt.title("DBSCAN Clustering")
plt.show(block=True)
```

Experiment 10 – Principal Component Analysis (PCA)

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
x = np.array([[2, 3], [3, 4], [4, 5], [5, 6], [6, 7]])
y = [0, 0, 1, 1, 1]
pca = PCA(n_components=1)
x_reduced = pca.fit_transform(x)
model = LogisticRegression().fit(x_reduced, y)
print("Predictions:", model.predict(x_reduced))
```

Experiment 11 – Single Layer Perceptron (AND, OR, XOR)

```
import numpy as np
from sklearn.neural_network import MLPClassifier
x = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = [0, 0, 0, 1]
y_or = [0, 1, 1, 1]
y_xor = [0, 1, 1, 0]
and_model = MLPClassifier(hidden_layer_sizes=(), max_iter=1000)
and_model.fit(x, y_and)
print("AND predictions:", and_model.predict(x))
or_model = MLPClassifier(hidden_layer_sizes=(), max_iter=1000)
or_model.fit(x, y_or)
print("OR predictions:", or_model.predict(x))
xor_model = MLPClassifier(hidden_layer_sizes=(), max_iter=1000)
xor_model.fit(x, y_xor)
print("XOR predictions:", xor_model.predict(x))
```

Experiment 12 – AdaBoost Algorithm

```
import numpy as np
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
x = np.array([[1], [2], [3], [4], [5]])
y = [0, 0, 1, 1, 1]
model = AdaBoostClassifier(
    base_estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=5
)
model.fit(x, y)
print("Predictions:", model.predict(x))
```